C# Lab Exam - 30 Practice Programs

24CS2017 C# and ASP .NET Programming Lab - Model Exam Questions (03.11.2026)

How to use this guide

- Every program below is a complete console program: create a project (dotnet new console, or Visual Studio > Console App), paste the code into Program.cs and run it.
- Each one was run on .NET 9 with the sample input shown under Output, and the results were checked.
- In the folder: ./run.sh <number> builds and runs any question (e.g. ./run.sh 15).
- Common input pattern: int[] a = Array.ConvertAll(Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries), int.Parse);

Quick revision table

No.	Question	Key idea
1	Generate the first 50 Fibonacci numbers.	Keep the last two numbers a, b; next = a + b, then shift. Use long: the 50th term (7778742049) overflows int.
2	Generate the first 50 prime numbers.	Test each n from 2 upward; n is prime if no i with i*i <= n divides it. Stop after 50 primes.
3	Check if a given string is a palindrome.	Two pointers from both ends moving inward; skip non-letters/digits and compare ignoring case.
4	Find the second largest number in an array without sorting.	One pass keeping largest and second (distinct). A bigger value pushes largest down to second.
5	Check if two strings are anagrams.	int[26] counts: +1 for each letter of the first string, -1 for the second; anagrams if all counts are 0.
6	Check if a string of parentheses/brackets is balanced.	Push opening brackets on a Stack; a closing bracket must match the popped top. Stack must be empty at the end.
7	Reverse the order of words in a sentence.	Split on spaces (RemoveEmptyEntries) and append the words from last to first with a StringBuilder.
8	Two Sum: indices of two numbers that add up to a target.	Dictionary value -> index. For each number check if (target - number) was already seen.
9	Count the frequency of each character in a string.	Dictionary<char,int> counts; a List keeps the order in which characters first appear.
10	Find the missing number in an array of 1 to N.	Expected sum N(N+1)/2 minus the actual sum is the missing number (use long to avoid overflow).
11	Find the first non-repeating character in a string.	For each character, count how often it appears with an inner loop; the first one with count 1 is the answer.
12	Remove duplicates from an array, keeping the original order.	Keep a result List; add a number only if result.Contains(x) is false, so the first occurrence wins.
13	Check if an integer is an Armstrong (narcissistic) number.	Sum of each digit raised to the number of digits equals the number (153 = 1^3 + 5^3 + 3^3). Use % 10 and / 10 to take digits off.
14	Binary search on a sorted array, iterative and recursive.	Compare with the middle element and keep only the half that can contain the key. mid = low + (high-low)/2.
15	Maximum-sum contiguous subarray (Kadane's algorithm).	Kadane: current = Max(a[i], current + a[i]) (extend or start fresh); best = Max(best, current).
16	Merge two sorted arrays without built-in sorting.	Two pointers i, j: copy the smaller front element each step, then copy the leftovers of either array.
17	Check if one string is a rotation of another.	s2 is a rotation of s1 exactly when lengths are equal and s2 occurs inside s1 + s1.
18	Basic string compression with counts (aabccccaaa -> a2b1c5a3).	Walk the string; for each run of the same character count it with an inner while loop, then add char + count.

19	Reverse a singly linked list in place.	prev/current/next pointers: save next, point current.Next back to prev, move both forward. prev is the new head.
20	Detect a cycle in a singly linked list (Floyd).	Floyd: slow moves 1 step, fast moves 2 steps; if they ever meet there is a cycle, if fast reaches null there is none.
21	Generate all permutations of a string.	Recursion: take each character in turn as the next letter (prefix + rest[i]), then permute what is left (rest.Remove(i, 1)). Print when nothing is left.
22	Length of the longest substring without repeating characters.	Start at every position and grow the substring until a character repeats (current.Contains); keep the longest.
23	Rotate an array to the right by k steps.	New array: the element at i moves to position (i + k) % n. The % n wraps it around and also handles k > n.
24	Intersection (common elements) of two integer arrays.	Two loops: for each element of the first array look for it in the second; add it once (skip if common already contains it).
25	Majority element (appears more than N/2 times).	For each element count how often it appears with an inner loop; if the count is more than n/2 it is the majority.
26	Thread-safe Singleton in C#.	Private constructor + static GetInstance(); lock(padlock) so only one thread at a time can check and create the instance.
27	LRU cache with O(1) Get and Put.	Dictionary key -> value for O(1) lookup; LinkedList of keys keeps usage order (front = most recent). A second Dictionary keeps each key's list node, so moving or removing it is O(1). When full, evict the key at the back.
28	Validate whether a binary tree is a valid BST.	Recursively pass (min, max) bounds: left child must be < node, right child > node, all the way down. Use long bounds.
29	Count the set bits (1s) in an integer.	Repeatedly look at the last binary digit (n % 2), count it if 1, then drop it (n / 2). Cast to uint so negatives work.
30	Producer-Consumer with BlockingCollection<T>.	BlockingCollection(3): producer Add() waits when full, then CompleteAdding(); consumer foreach over GetConsumingEnumerable() waits when empty and stops after the last item.

1. Generate the first 50 Fibonacci numbers.

Key idea: Keep the last two numbers a, b; next = a + b, then shift. Use long: the 50th term (7778742049) overflows int.

Complexity: O(n) time, O(1) space

Program

Q01_Fibonacci.cs

```
using System;

namespace Q01_Fibonacci
{
    class Program
    {
        static void Main()
        {
            // long is needed: the 50th Fibonacci number (7778742049) is larger than int.MaxValue
            long a = 0, b = 1;
            Console.WriteLine("First 50 Fibonacci numbers:");
            for (int i = 1; i <= 50; i++)
            {
                Console.Write(a + (i % 10 == 0 ? "\n" : " "));
                long next = a + b;
                a = b;
                b = next;
            }
        }
    }
}
```

Output

```
$ ./run.sh 1
First 50 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34
55 89 144 233 377 610 987 1597 2584 4181
6765 10946 17711 28657 46368 75025 121393 196418 317811 514229
832040 1346269 2178309 3524578 5702887 9227465 14930352 24157817 39088169 63245986
102334155 165580141 267914296 433494437 701408733 1134903170 1836311903 2971215073 4807526976
7778742049
```

2. Generate the first 50 prime numbers.

Key idea: Test each n from 2 upward; n is prime if no i with i*i <= n divides it. Stop after 50 primes.

Complexity: O(p * sqrt(p)) time where p is the 50th prime, O(1) space

Program

Q02_First50Primes.cs

```
using System;

namespace Q02_First50Primes
{
    class Program
    {
        static bool IsPrime(int n)
        {
            if (n < 2) return false;
            for (int i = 2; i * i <= n; i++)
                if (n % i == 0) return false;
            return true;
        }

        static void Main()
        {
            Console.WriteLine("First 50 prime numbers:");
            int count = 0;
            for (int n = 2; count < 50; n++)
            {
                if (IsPrime(n))
                {
                    count++;
                    Console.Write(n + (count % 10 == 0 ? "\n" : " "));
                }
            }
        }
    }
}
```

Output

```
$ ./run.sh 2
First 50 prime numbers:
2 3 5 7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
```

3. Check if a given string is a palindrome.

Key idea: Two pointers from both ends moving inward; skip non-letters/digits and compare ignoring case.

Complexity: O(n) time, O(1) space

Program

Q03_Palindrome.cs

```
using System;

namespace Q03_Palindrome
{
    class Program
    {
        // Two pointers from both ends; case and non-letter/digit characters are ignored
        static bool IsPalindrome(string s)
        {
            int i = 0, j = s.Length - 1;
            while (i < j)
            {
                if (!char.IsLetterOrDigit(s[i])) { i++; continue; }
                if (!char.IsLetterOrDigit(s[j])) { j--; continue; }
                if (char.ToLower(s[i]) != char.ToLower(s[j])) return false;
                i++;
                j--;
            }
            return true;
        }

        static void Main()
        {
            Console.Write("Enter a string: ");
            string s = Console.ReadLine();
            Console.WriteLine(IsPalindrome(s) ? $"\"{s}\" is a palindrome" : $"\"{s}\" is not a
palindrome");
        }
    }
}
```

Output

```
$ ./run.sh 3
Enter a string: Madam, I'm Adam
"Madam, I'm Adam" is a palindrome

$ ./run.sh 3
Enter a string: hello
"hello" is not a palindrome
```

4. Find the second largest number in an array without sorting.

Key idea: One pass keeping largest and second (distinct). A bigger value pushes largest down to second.

Complexity: O(n) time, O(1) space

Program

Q04_SecondLargest.cs

```
using System;

namespace Q04_SecondLargest
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter numbers separated by spaces: ");
            string[] parts = Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries);
            int[] arr = new int[parts.Length];
            for (int i = 0; i < parts.Length; i++) arr[i] = int.Parse(parts[i]);

            // One pass, no sorting: keep the largest and the second largest distinct value
            int largest = int.MinValue, second = int.MinValue;
            bool hasSecond = false;
            foreach (int x in arr)
            {
                if (x > largest)
                {
                    if (largest != int.MinValue) { second = largest; hasSecond = true; }
                    largest = x;
                }
                else if (x < largest && (!hasSecond || x > second))
                {
                    second = x;
                    hasSecond = true;
                }
            }

            Console.WriteLine("Largest: " + largest);
            Console.WriteLine(hasSecond ? "Second largest: " + second : "There is no second largest
(all values are equal)");
        }
    }
}
```

Output

```
$ ./run.sh 4
Enter numbers separated by spaces: 12 45 7 45 33 9
Largest: 45
Second largest: 33

$ ./run.sh 4
Enter numbers separated by spaces: 8 8 8
Largest: 8
There is no second largest (all values are equal)
```

5. Check if two strings are anagrams.

Key idea: int[26] counts: +1 for each letter of the first string, -1 for the second; anagrams if all counts are 0.

Complexity: O(n) time, O(1) space

Program

Q05_Anagram.cs

```
using System;

namespace Q05_Anagram
{
    class Program
    {
        // Count each letter: +1 for the first string, -1 for the second; all counts must end at 0
        static bool AreAnagrams(string a, string b)
        {
            int[] count = new int[26];
            int letters = 0;
            foreach (char c in a.ToLower())
                if (c >= 'a' && c <= 'z') { count[c - 'a']++; letters++; }
            foreach (char c in b.ToLower())
                if (c >= 'a' && c <= 'z') { count[c - 'a']--; letters--; }
            if (letters != 0) return false;
            foreach (int n in count)
                if (n != 0) return false;
            return true;
        }

        static void Main()
        {
            Console.Write("Enter first string: ");
            string a = Console.ReadLine();
            Console.Write("Enter second string: ");
            string b = Console.ReadLine();
            Console.WriteLine(AreAnagrams(a, b) ? "The strings are anagrams" : "The strings are not
anagrams");
        }
    }
}
```

Output

```
$ ./run.sh 5
Enter first string: Listen
Enter second string: Silent
The strings are anagrams

$ ./run.sh 5
Enter first string: Dormitory
Enter second string: dirty room
The strings are anagrams

$ ./run.sh 5
Enter first string: hello
Enter second string: world
The strings are not anagrams
```

6. Check if a string of parentheses/brackets is balanced.

Key idea: Push opening brackets on a Stack; a closing bracket must match the popped top. Stack must be empty at the end.

Complexity: O(n) time, O(n) space

Program

Q06_BalancedBrackets.cs

```
using System;
using System.Collections.Generic;

namespace Q06_BalancedBrackets
{
    class Program
    {
        static bool IsBalanced(string s)
        {
            Stack<char> stack = new Stack<char>();
            foreach (char c in s)
            {
                if (c == '(' || c == '[' || c == '{')
                    stack.Push(c);
                else if (c == ')' || c == ']' || c == '}')
                {
                    if (stack.Count == 0) return false;
                    char open = stack.Pop();
                    if ((c == ')' && open != '(') || (c == ']' && open != '[') || (c == '}' &&
open != '{'))
                        return false;
                }
            }
            return stack.Count == 0;   // nothing left open
        }

        static void Main()
        {
            Console.Write("Enter an expression: ");
            string s = Console.ReadLine();
            Console.WriteLine(IsBalanced(s) ? "Balanced" : "Not balanced");
        }
    }
}
```

Output

```
$ ./run.sh 6
Enter an expression: {[()()]}
Balanced

$ ./run.sh 6
Enter an expression: ([)]
Not balanced

$ ./run.sh 6
Enter an expression: ((
Not balanced
```

7. Reverse the order of words in a sentence.

Key idea: Split on spaces (RemoveEmptyEntries) and append the words from last to first with a StringBuilder.

Complexity: O(n) time, O(n) space

Program

Q07_ReverseWords.cs

```
using System;
using System.Text;

namespace Q07_ReverseWords
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter a sentence: ");
            string[] words = Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries);

            StringBuilder sb = new StringBuilder();
            for (int i = words.Length - 1; i >= 0; i--)
            {
                sb.Append(words[i]);
                if (i > 0) sb.Append(' ');
            }
            Console.WriteLine("Reversed: " + sb);
        }
    }
}
```

Output

```
$ ./run.sh 7
Enter a sentence: C# is a modern programming language
Reversed: language programming modern a is C#
```

8. Two Sum: indices of two numbers that add up to a target.

Key idea: Dictionary value -> index. For each number check if (target - number) was already seen.

Complexity: O(n) time, O(n) space

Program

Q08_TwoSum.cs

```
using System;
using System.Collections.Generic;

namespace Q08_TwoSum
{
    class Program
    {
        // For each number, check if (target - number) was already seen: O(n) with a dictionary
        static int[] TwoSum(int[] nums, int target)
        {
            Dictionary<int, int> seen = new Dictionary<int, int>();   // value -> index
            for (int i = 0; i < nums.Length; i++)
            {
                int need = target - nums[i];
                if (seen.ContainsKey(need))
                    return new int[] { seen[need], i };
                seen[nums[i]] = i;
            }
            return null;
        }

        static void Main()
        {
            Console.Write("Enter numbers separated by spaces: ");
            string[] parts = Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries);
            int[] nums = Array.ConvertAll(parts, int.Parse);
            Console.Write("Enter target sum: ");
            int target = int.Parse(Console.ReadLine());

            int[] result = TwoSum(nums, target);
            if (result == null)
                Console.WriteLine("No two numbers add up to " + target);
            else
                Console.WriteLine($"Indices: [{result[0]}, {result[1]}]  ({nums[result[0]]} +
{nums[result[1]]} = {target})");
        }
    }
}
```

Output

```
$ ./run.sh 8
Enter numbers separated by spaces: 2 7 11 15
Enter target sum: 9
Indices: [0, 1]  (2 + 7 = 9)

$ ./run.sh 8
Enter numbers separated by spaces: 3 2 4
Enter target sum: 6
Indices: [1, 2]  (2 + 4 = 6)

$ ./run.sh 8
Enter numbers separated by spaces: 1 2 3
Enter target sum: 10
No two numbers add up to 10
```

9. Count the frequency of each character in a string.

Key idea: Dictionary<char,int> counts; a List keeps the order in which characters first appear.

Complexity: O(n) time, O(k) space for k distinct characters

Program

Q09_CharFrequency.cs

```
using System;
using System.Collections.Generic;

namespace Q09_CharFrequency
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter a string: ");
            string s = Console.ReadLine();

            Dictionary<char, int> freq = new Dictionary<char, int>();
            List<char> order = new List<char>();          // remembers first-appearance order
            foreach (char c in s)
            {
                if (c == ' ') continue;
                if (freq.ContainsKey(c))
                    freq[c]++;
                else
                {
                    freq[c] = 1;
                    order.Add(c);
                }
            }

            Console.WriteLine("Character frequencies:");
            foreach (char c in order)
                Console.WriteLine($"'{c}' : {freq[c]}");
        }
    }
}
```

Output

```
$ ./run.sh 9
Enter a string: hello world
Character frequencies:
'h' : 1
'e' : 1
'l' : 3
'o' : 2
'w' : 1
'r' : 1
'd' : 1
```

10. Find the missing number in an array of 1 to N.

Key idea: Expected sum N(N+1)/2 minus the actual sum is the missing number (use long to avoid overflow).

Complexity: O(n) time, O(1) space

Program

Q10_MissingNumber.cs

```
using System;

namespace Q10_MissingNumber
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter N: ");
            int n = int.Parse(Console.ReadLine());
            Console.Write($"Enter the {n - 1} numbers (1 to {n}, one missing): ");
            string[] parts = Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries);

            // Sum of 1..N is N(N+1)/2; whatever is missing from that total is the answer
            long expected = (long)n * (n + 1) / 2;
            long actual = 0;
            foreach (string p in parts) actual += int.Parse(p);

            Console.WriteLine("Missing number: " + (expected - actual));
        }
    }
}
```

Output

```
$ ./run.sh 10
Enter N: 8
Enter the 7 numbers (1 to 8, one missing): 1 2 4 5 6 7 8
Missing number: 3
```

11. Find the first non-repeating character in a string.

Key idea: For each character, count how often it appears with an inner loop; the first one with count 1 is the answer.

Complexity: O(n^2) time, O(1) space

Program

Q11_FirstNonRepeating.cs

```
using System;

namespace Q11_FirstNonRepeating
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter a string: ");
            string s = Console.ReadLine();

            // For each character, count how many times it appears in the whole string
            for (int i = 0; i < s.Length; i++)
            {
                int count = 0;
                for (int j = 0; j < s.Length; j++)
                    if (s[j] == s[i]) count++;

                if (count == 1)
                {
                    Console.WriteLine("First non-repeating character: '" + s[i] + "' at index " +
i);
                    return;
                }
            }
            Console.WriteLine("Every character repeats");
        }
    }
}
```

Output

```
$ ./run.sh 11
Enter a string: swiss
First non-repeating character: 'w' at index 1

$ ./run.sh 11
Enter a string: aabb
Every character repeats
```

12. Remove duplicates from an array, keeping the original order.

Key idea: Keep a result List; add a number only if result.Contains(x) is false, so the first occurrence wins.

Complexity: O(n^2) time, O(n) space

Program

Q12_RemoveDuplicates.cs

```
using System;
using System.Collections.Generic;

namespace Q12_RemoveDuplicates
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter numbers separated by spaces: ");
            string[] parts = Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries);

            List<int> result = new List<int>();
            foreach (string p in parts)
            {
                int x = int.Parse(p);
                if (!result.Contains(x))      // add only if we have not added it before
                    result.Add(x);
            }
            Console.WriteLine("Without duplicates: " + string.Join(" ", result));
        }
    }
}
```

Output

```
$ ./run.sh 12
Enter numbers separated by spaces: 4 2 4 7 2 9 7 1
Without duplicates: 4 2 7 9 1
```

13. Check if an integer is an Armstrong (narcissistic) number.

Key idea: Sum of each digit raised to the number of digits equals the number (153 = 1^3 + 5^3 + 3^3). Use % 10 and / 10 to take digits off.

Complexity: O(d) time for d digits, O(1) space

Program

Q13_Armstrong.cs

```
using System;

namespace Q13_Armstrong
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter an integer: ");
            int n = int.Parse(Console.ReadLine());

            int digits = n.ToString().Length;     // e.g. 153 has 3 digits
            int sum = 0;
            int temp = n;
            while (temp > 0)
            {
                int d = temp % 10;                // last digit
                sum += (int)Math.Pow(d, digits);  // digit raised to the number of digits
                temp = temp / 10;                 // remove the last digit
            }

            if (sum == n)
                Console.WriteLine(n + " is an Armstrong number");
            else
                Console.WriteLine(n + " is not an Armstrong number");
        }
    }
}
```

Output

```
$ ./run.sh 13
Enter an integer: 153
153 is an Armstrong number

$ ./run.sh 13
Enter an integer: 125
125 is not an Armstrong number
```

14. Binary search on a sorted array, iterative and recursive.

Key idea: Compare with the middle element and keep only the half that can contain the key. mid = low + (high-low)/2.

Complexity: O(log n) time; O(1) space iterative, O(log n) stack recursive

Program

Q14_BinarySearch.cs

```
using System;

namespace Q14_BinarySearch
{
    class Program
    {
        static int SearchIterative(int[] a, int key)
        {
            int low = 0, high = a.Length - 1;
            while (low <= high)
            {
                int mid = low + (high - low) / 2;
                if (a[mid] == key) return mid;
                if (a[mid] < key) low = mid + 1;
                else high = mid - 1;
            }
            return -1;
        }

        static int SearchRecursive(int[] a, int key, int low, int high)
        {
            if (low > high) return -1;
            int mid = low + (high - low) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) return SearchRecursive(a, key, mid + 1, high);
            return SearchRecursive(a, key, low, mid - 1);
        }

        static void Main()
        {
            Console.Write("Enter sorted numbers separated by spaces: ");
            int[] a = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);
            Console.Write("Enter number to search: ");
            int key = int.Parse(Console.ReadLine());

            int i1 = SearchIterative(a, key);
            int i2 = SearchRecursive(a, key, 0, a.Length - 1);
            Console.WriteLine(i1 >= 0 ? $"Iterative: found {key} at index {i1}" : $"Iterative: {key}
not found");
            Console.WriteLine(i2 >= 0 ? $"Recursive: found {key} at index {i2}" : $"Recursive: {key}
not found");
        }
    }
}
```

Output

```
$ ./run.sh 14
Enter sorted numbers separated by spaces: 2 5 8 12 16 23 38 56 72 91
Enter number to search: 23
Iterative: found 23 at index 5
Recursive: found 23 at index 5

$ ./run.sh 14
Enter sorted numbers separated by spaces: 2 5 8 12 16 23 38 56 72 91
Enter number to search: 40
Iterative: 40 not found
Recursive: 40 not found
```

15. Maximum-sum contiguous subarray (Kadane's algorithm).

Key idea: Kadane: current = Max(a[i], current + a[i]) (extend or start fresh); best = Max(best, current).

Complexity: O(n) time, O(1) space

Program

Q15_Kadane.cs

```
using System;

namespace Q15_Kadane
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter numbers separated by spaces: ");
            int[] a = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);

            // Kadane: at each number, either add it to the current subarray or start fresh from it
            int current = a[0];
            int best = a[0];
            for (int i = 1; i < a.Length; i++)
            {
                current = Math.Max(a[i], current + a[i]);
                best = Math.Max(best, current);
            }
            Console.WriteLine("Maximum subarray sum: " + best);
        }
    }
}
```

Output

```
$ ./run.sh 15
Enter numbers separated by spaces: -2 1 -3 4 -1 2 1 -5 4
Maximum subarray sum: 6

$ ./run.sh 15
Enter numbers separated by spaces: -3 -1 -2
Maximum subarray sum: -1
```

16. Merge two sorted arrays without built-in sorting.

Key idea: Two pointers i, j: copy the smaller front element each step, then copy the leftovers of either array.

Complexity: O(n + m) time, O(n + m) space

Program

Q16_MergeSortedArrays.cs

```
using System;

namespace Q16_MergeSortedArrays
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter first sorted array: ");
            int[] a = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);
            Console.Write("Enter second sorted array: ");
            int[] b = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);

            int[] result = new int[a.Length + b.Length];
            int i = 0, j = 0, k = 0;

            // Compare the front of both arrays and copy the smaller one
            while (i < a.Length && j < b.Length)
            {
                if (a[i] <= b[j])
                {
                    result[k] = a[i];
                    i++;
                }
                else
                {
                    result[k] = b[j];
                    j++;
                }
                k++;
            }
            // Copy whatever is left in either array
            while (i < a.Length) { result[k] = a[i]; i++; k++; }
            while (j < b.Length) { result[k] = b[j]; j++; k++; }

            Console.WriteLine("Merged: " + string.Join(" ", result));
        }
    }
}
```

Output

```
$ ./run.sh 16
Enter first sorted array: 1 4 7 10
Enter second sorted array: 2 3 8 12 15
Merged: 1 2 3 4 7 8 10 12 15
```

17. Check if one string is a rotation of another.

Key idea: s2 is a rotation of s1 exactly when lengths are equal and s2 occurs inside s1 + s1.

Complexity: O(n) time with a linear search (Contains), O(n) space

Program

Q17_StringRotation.cs

```
using System;

namespace Q17_StringRotation
{
    class Program
    {
        // If s2 is a rotation of s1, then s2 always appears inside s1 + s1
        static bool IsRotation(string s1, string s2)
        {
            return s1.Length == s2.Length && s1.Length > 0 && (s1 + s1).Contains(s2);
        }

        static void Main()
        {
            Console.Write("Enter first string: ");
            string s1 = Console.ReadLine();
            Console.Write("Enter second string: ");
            string s2 = Console.ReadLine();
            Console.WriteLine(IsRotation(s1, s2)
                ? $"\"{s2}\" is a rotation of \"{s1}\""
                : $"\"{s2}\" is not a rotation of \"{s1}\"");
        }
    }
}
```

Output

```
$ ./run.sh 17
Enter first string: waterbottle
Enter second string: erbottlewat
"erbottlewat" is a rotation of "waterbottle"

$ ./run.sh 17
Enter first string: hello
Enter second string: olleh
"olleh" is not a rotation of "hello"
```

18. Basic string compression with counts (aabccccccaaa -> a2b1c5a3).

Key idea: Walk the string; for each run of the same character count it with an inner while loop, then add char + count.

Complexity: O(n) time, O(n) space

Program

Q18_StringCompression.cs

```
using System;

namespace Q18_StringCompression
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter a string: ");
            string s = Console.ReadLine();

            string result = "";
            int i = 0;
            while (i < s.Length)
            {
                char c = s[i];
                int count = 0;
                while (i < s.Length && s[i] == c)   // count this run of the same character
                {
                    count++;
                    i++;
                }
                result = result + c + count;        // e.g. "a" + 2 -> "a2"
            }
            Console.WriteLine("Compressed: " + result);
        }
    }
}
```

Output

```
$ ./run.sh 18
Enter a string: aabccccccaaa
Compressed: a2b1c5a3

$ ./run.sh 18
Enter a string: abc
Compressed: a1b1c1
```

19. Reverse a singly linked list in place.

Key idea: prev/current/next pointers: save next, point current.Next back to prev, move both forward. prev is the new head.

Complexity: O(n) time, O(1) space

Program

Q19_ReverseLinkedList.cs

```
using System;

namespace Q19_ReverseLinkedList
{
    class Node
    {
        public int Data;
        public Node Next;
        public Node(int data) { Data = data; }
    }

    class Program
    {
        // Walk the list once and turn every Next pointer around
        static Node Reverse(Node head)
        {
            Node prev = null, current = head;
            while (current != null)
            {
                Node next = current.Next;
                current.Next = prev;
                prev = current;
                current = next;
            }
            return prev;
        }

        static void Print(Node head)
        {
            for (Node n = head; n != null; n = n.Next)
                Console.Write(n.Data + (n.Next != null ? " -> " : " -> null\n"));
        }

        static void Main()
        {
            Console.Write("Enter list values separated by spaces: ");
            string[] parts = Console.ReadLine().Split(' ', StringSplitOptions.RemoveEmptyEntries);
            Node head = null, tail = null;
            foreach (string p in parts)
            {
                Node node = new Node(int.Parse(p));
                if (head == null) head = tail = node;
                else { tail.Next = node; tail = node; }
            }

            Console.Write("Original: ");
            Print(head);
            head = Reverse(head);
            Console.Write("Reversed: ");
            Print(head);
        }
    }
}
```

Output

```
$ ./run.sh 19
Enter list values separated by spaces: 10 20 30 40 50
```

```
Original: 10 -> 20 -> 30 -> 40 -> 50 -> null
Reversed: 50 -> 40 -> 30 -> 20 -> 10 -> null
```

20. Detect a cycle in a singly linked list (Floyd).

Key idea: Floyd: slow moves 1 step, fast moves 2 steps; if they ever meet there is a cycle, if fast reaches null there is none.

Complexity: O(n) time, O(1) space

Program

Q20_DetectCycle.cs

```
using System;

namespace Q20_DetectCycle
{
    class Node
    {
        public int Data;
        public Node Next;
        public Node(int data) { Data = data; }
    }

    class Program
    {
        // Floyd: slow moves 1 step, fast moves 2 steps. If they ever meet, there is a cycle.
        static bool HasCycle(Node head)
        {
            Node slow = head, fast = head;
            while (fast != null && fast.Next != null)
            {
                slow = slow.Next;
                fast = fast.Next.Next;
                if (slow == fast) return true;
            }
            return false;     // fast reached the end, so no cycle
        }

        static void Main()
        {
            // List without a cycle: 1 -> 2 -> 3 -> 4 -> null
            Node a = new Node(1);
            a.Next = new Node(2);
            a.Next.Next = new Node(3);
            a.Next.Next.Next = new Node(4);
            Console.WriteLine("List 1 -> 2 -> 3 -> 4 -> null : " + (HasCycle(a) ? "Cycle found" :
"No cycle"));

            // List with a cycle: 1 -> 2 -> 3 -> 4 -> back to 2
            Node b = new Node(1);
            b.Next = new Node(2);
            b.Next.Next = new Node(3);
            b.Next.Next.Next = new Node(4);
            b.Next.Next.Next.Next = b.Next;
            Console.WriteLine("List 1 -> 2 -> 3 -> 4 -> back to 2 : " + (HasCycle(b) ? "Cycle found"
: "No cycle"));
        }
    }
}
```

Output

```
$ ./run.sh 20
List 1 -> 2 -> 3 -> 4 -> null : No cycle
List 1 -> 2 -> 3 -> 4 -> back to 2 : Cycle found
```

21. Generate all permutations of a string.

Key idea: Recursion: take each character in turn as the next letter (prefix + rest[i]), then permute what is left (rest.Remove(i, 1)). Print when nothing is left.

Complexity: O(n * n!) time, O(n) recursion depth

Program

Q21_Permutations.cs

```
using System;

namespace Q21_Permutations
{
    class Program
    {
        // Take each character in turn as the next letter, then permute the letters that are left
        static void Permute(string prefix, string rest)
        {
            if (rest.Length == 0)
            {
                Console.WriteLine(prefix);
                return;
            }
            for (int i = 0; i < rest.Length; i++)
            {
                string remaining = rest.Remove(i, 1);   // rest without the character at i
                Permute(prefix + rest[i], remaining);
            }
        }

        static void Main()
        {
            Console.Write("Enter a string: ");
            string s = Console.ReadLine();
            Console.WriteLine("Permutations:");
            Permute("", s);
        }
    }
}
```

Output

```
$ ./run.sh 21
Enter a string: abc
Permutations:
abc
acb
bac
bca
cab
cba
```

22. Length of the longest substring without repeating characters.

Key idea: Start at every position and grow the substring until a character repeats (current.Contains); keep the longest.

Complexity: O(n^3) worst case, fine for exam-size input

Program

Q22_LongestUniqueSubstring.cs

```
using System;

namespace Q22_LongestUniqueSubstring
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter a string: ");
            string s = Console.ReadLine();

            string longest = "";
            // Start at every position and grow the substring until a character repeats
            for (int i = 0; i < s.Length; i++)
            {
                string current = "";
                for (int j = i; j < s.Length; j++)
                {
                    if (current.Contains(s[j])) break;   // repeat found, stop growing
                    current = current + s[j];
                }
                if (current.Length > longest.Length)
                    longest = current;
            }
            Console.WriteLine("Longest substring without repeating characters: \"" + longest +
"\"");
            Console.WriteLine("Length: " + longest.Length);
        }
    }
}
```

Output

```
$ ./run.sh 22
Enter a string: abcabcbb
Longest substring without repeating characters: "abc"
Length: 3

$ ./run.sh 22
Enter a string: pwwkew
Longest substring without repeating characters: "wke"
Length: 3
```

23. Rotate an array to the right by k steps.

Key idea: New array: the element at i moves to position (i + k) % n. The % n wraps it around and also handles k > n.

Complexity: O(n) time, O(n) space

Program

Q23_RotateArray.cs

```
using System;

namespace Q23_RotateArray
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter numbers separated by spaces: ");
            int[] a = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);
            Console.Write("Rotate right by k = ");
            int k = int.Parse(Console.ReadLine());

            int n = a.Length;
            int[] result = new int[n];
            // Each element moves k places to the right; % n wraps it around to the front
            for (int i = 0; i < n; i++)
                result[(i + k) % n] = a[i];

            Console.WriteLine("Rotated: " + string.Join(" ", result));
        }
    }
}
```

Output

```
$ ./run.sh 23
Enter numbers separated by spaces: 1 2 3 4 5 6 7
Rotate right by k = 3
Rotated: 5 6 7 1 2 3 4

$ ./run.sh 23
Enter numbers separated by spaces: 1 2 3 4 5 6 7
Rotate right by k = 10
Rotated: 5 6 7 1 2 3 4
```

24. Intersection (common elements) of two integer arrays.

Key idea: Two loops: for each element of the first array look for it in the second; add it once (skip if common already contains it).

Complexity: O(n * m) time, O(k) space

Program

Q24_ArrayIntersection.cs

```
using System;
using System.Collections.Generic;

namespace Q24_ArrayIntersection
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter first array: ");
            int[] a = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);
            Console.Write("Enter second array: ");
            int[] b = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);

            List<int> common = new List<int>();
            for (int i = 0; i < a.Length; i++)
            {
                for (int j = 0; j < b.Length; j++)
                {
                    if (a[i] == b[j] && !common.Contains(a[i]))   // in both arrays, not added yet
                    {
                        common.Add(a[i]);
                        break;
                    }
                }
            }
            Console.WriteLine(common.Count > 0 ? "Common elements: " + string.Join(" ", common) :
"No common elements");
        }
    }
}
```

Output

```
$ ./run.sh 24
Enter first array: 1 2 2 3 4 5
Enter second array: 2 2 4 6 8
Common elements: 2 4
```

25. Majority element (appears more than N/2 times).

Key idea: For each element count how often it appears with an inner loop; if the count is more than n/2 it is the majority.

Complexity: O(n^2) time, O(1) space

Program

Q25_MajorityElement.cs

```
using System;

namespace Q25_MajorityElement
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter numbers separated by spaces: ");
            int[] a = Array.ConvertAll(Console.ReadLine().Split(' ',
StringSplitOptions.RemoveEmptyEntries), int.Parse);

            // Count how often each number appears; the majority appears more than n/2 times
            for (int i = 0; i < a.Length; i++)
            {
                int count = 0;
                for (int j = 0; j < a.Length; j++)
                    if (a[j] == a[i]) count++;

                if (count > a.Length / 2)
                {
                    Console.WriteLine("Majority element: " + a[i] + " (appears " + count + " of " +
a.Length + " times)");
                    return;
                }
            }
            Console.WriteLine("No majority element");
        }
    }
}
```

Output

```
$ ./run.sh 25
Enter numbers separated by spaces: 2 2 1 1 1 2 2
Majority element: 2 (appears 4 of 7 times)

$ ./run.sh 25
Enter numbers separated by spaces: 1 2 3 1
No majority element
```

26. Thread-safe Singleton in C#.

Key idea: Private constructor + static GetInstance(); lock(padlock) so only one thread at a time can check and create the instance.

Complexity: O(1) per call; constructor runs exactly once

Program

Q26_ThreadSafeSingleton.cs

```
using System;
using System.Threading.Tasks;

namespace Q26_ThreadSafeSingleton
{
    class Singleton
    {
        private static Singleton instance;                // the one and only object
        private static readonly object padlock = new object();
        public static int CreatedCount = 0;

        private Singleton()                               // private: nobody outside can use "new"
        {
            CreatedCount++;
        }

        public static Singleton GetInstance()
        {
            lock (padlock)                                // only one thread at a time can enter
            {
                if (instance == null)
                    instance = new Singleton();
                return instance;
            }
        }
    }

    class Program
    {
        static void Main()
        {
            // 20 threads ask for the instance at the same time
            Parallel.For(0, 20, i => Singleton.GetInstance());
            Console.WriteLine("20 threads asked for the instance");
            Console.WriteLine("Constructor ran " + Singleton.CreatedCount + " time(s)");

            Singleton s1 = Singleton.GetInstance();
            Singleton s2 = Singleton.GetInstance();
            Console.WriteLine("s1 and s2 are the same object: " + (s1 == s2));
        }
    }
}
```

Output

```
$ ./run.sh 26
20 threads asked for the instance
Constructor ran 1 time(s)
s1 and s2 are the same object: True
```

27. LRU cache with O(1) Get and Put.

Key idea: Dictionary key -> value for O(1) lookup; LinkedList of keys keeps usage order (front = most recent). A second Dictionary keeps each key's list node, so moving or removing it is O(1). When full, evict the key at the back.

Complexity: O(1) per Get/Put, O(capacity) space

Program

Q27_LRUCache.cs

```
using System;
using System.Collections.Generic;

namespace Q27_LRUCache
{
    class LRUCache
    {
        int capacity;
        Dictionary<int, int> values = new Dictionary<int, int>();                       // key ->
value
        Dictionary<int, LinkedListNode<int>> nodes = new Dictionary<int, LinkedListNode<int>>(); //
key -> its place in the list
        LinkedList<int> order = new LinkedList<int>();                                  // keys,
most recent at the front

        public LRUCache(int capacity)
        {
            this.capacity = capacity;
        }

        public int Get(int key)
        {
            if (!values.ContainsKey(key)) return -1;
            MoveToFront(key);                    // it was just used
            return values[key];
        }

        public void Put(int key, int value)
        {
            if (values.ContainsKey(key))
            {
                values[key] = value;             // update and mark as used
                MoveToFront(key);
                return;
            }
            if (values.Count == capacity)        // full: remove the least recently used (back of
the list)
            {
                int lru = order.Last.Value;
                order.RemoveLast();
                values.Remove(lru);
                nodes.Remove(lru);
                Console.WriteLine("  evicted key " + lru);
            }
            values[key] = value;
            nodes[key] = order.AddFirst(key);
        }

        void MoveToFront(int key)
        {
            order.Remove(nodes[key]);            // O(1) because we already have the node
            nodes[key] = order.AddFirst(key);
        }
    }

    class Program
    {
        static void Main()
```

```
        {
            LRUCache cache = new LRUCache(2);
            Console.WriteLine("Capacity = 2");
            Console.WriteLine("Put(1, 1)"); cache.Put(1, 1);
            Console.WriteLine("Put(2, 2)"); cache.Put(2, 2);
            Console.WriteLine("Get(1) = " + cache.Get(1));
            Console.WriteLine("Put(3, 3)"); cache.Put(3, 3);
            Console.WriteLine("Get(2) = " + cache.Get(2));
            Console.WriteLine("Put(4, 4)"); cache.Put(4, 4);
            Console.WriteLine("Get(1) = " + cache.Get(1));
            Console.WriteLine("Get(3) = " + cache.Get(3));
            Console.WriteLine("Get(4) = " + cache.Get(4));
        }
    }
}
```

Output

```
$ ./run.sh 27
Capacity = 2
Put(1, 1)
Put(2, 2)
Get(1) = 1
Put(3, 3)
  evicted key 2
Get(2) = -1
Put(4, 4)
  evicted key 1
Get(1) = -1
Get(3) = 3
Get(4) = 4
```

28. Validate whether a binary tree is a valid BST.

Key idea: Recursively pass (min, max) bounds: left child must be < node, right child > node, all the way down. Use long bounds.

Complexity: O(n) time, O(h) recursion depth

Program

Q28_ValidateBST.cs

```
using System;

namespace Q28_ValidateBST
{
    class Node
    {
        public int Val;
        public Node Left, Right;
        public Node(int val) { Val = val; }
    }

    class Program
    {
        // Every node must be bigger than min and smaller than max
        static bool IsBST(Node node, long min, long max)
        {
            if (node == null) return true;
            if (node.Val <= min || node.Val >= max) return false;
            return IsBST(node.Left, min, node.Val) && IsBST(node.Right, node.Val, max);
        }

        static void Main()
        {
            //        5
            //      /   \
            //     3     8
            //    / \   / \
            //   2   4 7   9
            Node t1 = new Node(5);
            t1.Left = new Node(3);
            t1.Right = new Node(8);
            t1.Left.Left = new Node(2);
            t1.Left.Right = new Node(4);
            t1.Right.Left = new Node(7);
            t1.Right.Right = new Node(9);
            Console.WriteLine("Tree 1: " + (IsBST(t1, long.MinValue, long.MaxValue) ? "Valid BST" :
"Not a valid BST"));

            //        5
            //      /   \
            //     1     4      <- 4 is on the right of 5 but smaller than 5
            //          / \
            //         3   6
            Node t2 = new Node(5);
            t2.Left = new Node(1);
            t2.Right = new Node(4);
            t2.Right.Left = new Node(3);
            t2.Right.Right = new Node(6);
            Console.WriteLine("Tree 2: " + (IsBST(t2, long.MinValue, long.MaxValue) ? "Valid BST" :
"Not a valid BST"));
        }
    }
}
```

Output

```
$ ./run.sh 28
Tree 1: Valid BST
Tree 2: Not a valid BST
```

29. Count the set bits (1s) in an integer.

Key idea: Repeatedly look at the last binary digit (n % 2), count it if 1, then drop it (n / 2). Cast to uint so negatives work.

Complexity: O(32) = O(1) time, O(1) space

Program

Q29_CountSetBits.cs

```
using System;

namespace Q29_CountSetBits
{
    class Program
    {
        static void Main()
        {
            Console.Write("Enter an integer: ");
            int number = int.Parse(Console.ReadLine());
            Console.WriteLine("Binary: " + Convert.ToString(number, 2));

            uint n = (uint)number;   // uint so negative numbers also work
            int count = 0;
            while (n > 0)
            {
                if (n % 2 == 1) count++;   // last binary digit is 1
                n = n / 2;                 // drop the last binary digit
            }
            Console.WriteLine("Number of set bits: " + count);
        }
    }
}
```

Output

```
$ ./run.sh 29
Enter an integer: 29
Binary: 11101
Number of set bits: 4

$ ./run.sh 29
Enter an integer: -1
Binary: 11111111111111111111111111111111
Number of set bits: 32
```

30. Producer-Consumer with BlockingCollection<T>.

Key idea: BlockingCollection(3): producer Add() waits when full, then CompleteAdding(); consumer foreach over GetConsumingEnumerable() waits when empty and stops after the last item.

Complexity: Producer and consumer run on two threads at the same time

Program

Q30_ProducerConsumer.cs

```
using System;
using System.Collections.Concurrent;
using System.Threading;

namespace Q30_ProducerConsumer
{
    class Program
    {
        static void Main()
        {
            // A thread-safe queue that holds at most 3 items
            BlockingCollection<int> queue = new BlockingCollection<int>(3);

            Thread producer = new Thread(() =>
            {
                for (int i = 1; i <= 8; i++)
                {
                    Console.WriteLine("Produced " + i);
                    queue.Add(i);              // waits if the queue is full
                    Thread.Sleep(30);
                }
                queue.CompleteAdding();        // no more items are coming
                Console.WriteLine("Producer finished");
            });

            Thread consumer = new Thread(() =>
            {
                int sum = 0;
                foreach (int item in queue.GetConsumingEnumerable())   // waits if the queue is
empty
                {
                    Console.WriteLine("        Consumed " + item);
                    sum += item;
                    Thread.Sleep(80);
                }
                Console.WriteLine("Consumer finished, sum of items = " + sum);
            });

            producer.Start();
            consumer.Start();
            producer.Join();                   // wait for both threads to finish
            consumer.Join();
        }
    }
}
```

Output

```
$ ./run.sh 30
Produced 1
        Consumed 1
Produced 2
Produced 3
        Consumed 2
Produced 4
Produced 5
Produced 6
        Consumed 3
Produced 7
```

```
        Consumed 4
Produced 8
        Consumed 5
Producer finished
        Consumed 6
        Consumed 7
        Consumed 8
Consumer finished, sum of items = 36
```